

A Hybrid Tracker Combining Kernelized Correlation Filter and Tracking–Learning–Detection for UAV Tracking: Comparison with Kernelized Correlation Filter, Discriminative Correlation Filter with Channel and Spatial Reliability, and Tracking–Learning–Detection

Frank Xu

May 4, 2026

Contents

1	Introduction	2
2	Literature Review	3
2.1	KCF: High-Speed Correlation Filtering	4
2.2	CSRT: Reliability-Aware Correlation Filtering	5
2.3	TLD: Long-Term Recovery Through Tracking, Learning, and Detection . .	5
2.4	Why a Hybrid KCF–TLD Design Is Needed	6
3	Evaluation	7
4	Methodology	7
4.1	Overview of the Proposed Tracker	7
4.2	Tracking and Recovery Stages	9
4.2.1	Kernelized Correlation Filter stage	9

4.2.2	Tracking–Learning–Detection recovery stage	9
4.3	Thresholds, Decision Logic, and Detailed Flowcharts	10
4.3.1	Detailed main flowchart	11
4.3.2	Detailed recovery decision flow	11
5	Results and Analysis	13
5.1	Experimental Setup	13
5.2	Per-Sequence Interpretation	13
5.3	Aggregate Comparison	14
5.4	Benchmark Figures	15
6	Discussion	19
7	Conclusion	21

1 Introduction

Although discriminative visual tracking has developed rapidly and now includes correlation-filter variants, Siamese architectures, and stronger optimisation-based trackers, it remains useful to return to a smaller set of classical methods when the aim is to compare design logic rather than to list the strongest leaderboard entries^[2,6-7,18]. For UAV-style tasks in particular, this kind of comparison is still meaningful, because speed, localisation quality, and post-failure recovery are rarely improved to the same degree by one method alone^[9-10]. Among the classical representatives considered here, the Kernelized Correlation Filter (KCF) proposed by Henriques et al.^[12] achieves high efficiency by combining circulant matrices, ridge regression, and the kernel trick in the Fourier domain; its background can also be understood in relation to earlier correlation-filter work such as MOSSE^[4]. The Discriminative Correlation Filter with Channel and Spatial Reliability (CSRT) proposed by Lukežić et al.^[17] extends the same broad discriminative framework by introducing channel reliability and spatial reliability so that robustness can be improved. Tracking–Learning–Detection (TLD) proposed by Kalal et al.^[14] follows a different route by combining tracking, learning, and detection, and by updating its detector online to adapt to appearance variation.

Although these are all classic discriminative tracker algorithms, the differences between

them are very obvious. Whether it is their basic principles, performance priorities, or operational behavior and recovery capabilities, they are completely different. For those who are not familiar with the field of tracking, it is often difficult to understand these differences just from the names in benchmark tests. A tracker that looks attractive because of its speed may become weak or even completely lost after occlusion. A more precise tracker requires more computation, not to mention it takes too much time. A tracker that can re-detect the target is still too slow to remain active as the main tracker throughout the sequence. This is the difference between them.

This paper therefore analyses KCF, CSRT, and TLD first at the level of principle and algorithm design, with the aim of giving a clearer and more accessible comparison of their differences. On that basis, it then proposes a hybrid tracker that combines KCF and TLD, referred to below as KCF-TLD. The motivation does not lie in simply putting two classical methods together. Rather, it comes from a practical imbalance between them: KCF is very fast during ordinary tracking, but it becomes weak after target loss, whereas TLD offers recovery ability but is too slow to remain active as the main tracker throughout the whole sequence. The proposed design therefore assigns them different roles, so that KCF handles routine high-speed tracking while TLD is called only when recovery becomes necessary.

2 Literature Review

Current research on discriminative visual tracking has expanded in many directions, but at the basic mechanism level, it is still useful to distinguish three directly related approaches^[18]. The first is kernelized correlation filters, with the clearest representative algorithm being KCF. The second type of algorithm still falls within the correlation filter framework, but enhances robustness through channel reliability and spatial reliability; the representative of this type is CSRT. The third approach combines tracking, learning, and detection, placing more emphasis on long-term recovery methods, namely TLD. In fact, later research has not eliminated these distinctions, but has continuously deepened them through improvements in scale handling, spatial regularization, reliable patch selection, complementary learners, twin matching, and stronger appearance modeling^[1-3,6-8,10-11,15-16,21].

This review therefore concerns the three major original papers of these algorithms. The main purpose of the review is to cite relevant research, to summarize the contributions of each original algorithm, their strengths and weaknesses, and the reasons why such differences are more important for later KCF–TLD designs.

2.1 KCF: High-Speed Correlation Filtering

KCF can be understood in a relatively direct way as a correlation-filter tracker built around an efficient discriminative learning process. As Henriques et al. explain^[12], once cyclic shifts of a target patch are recycled into a dense training set and the circulant structure is processed in the Fourier domain, the cost of the corresponding ridge-regression problem is reduced substantially. In that sense, KCF extends the earlier correlation-filter line represented by MOSSE while also making the formulation mathematically more concise and flexible^[4]. The importance of KCF therefore lies not only in empirical speed, but also in the fact that its speed follows from a remarkably transparent mathematical construction.

In its standard form, the regression objective can be written as

$$w = (X^T X + \lambda I)^{-1} X^T Y,$$

where X is the sample matrix, Y is the regression target, and λ is the regularisation parameter. Combining the kernel trick with the diagonalisation property of circulant matrices under the discrete Fourier transform, KCF reduces computations and can be performed at very high frame rate.

Its advantages are also fairly easy to understand. KCF has high computational efficiency, and through FFT-based filtering it can maintain fast short-term tracking under ordinary conditions. The use of kernel mapping also helps it remain reasonably robust against changes in pose, illumination variation, and limited scale fluctuation. At the same time, its limitations are equally clear. KCF performs much better when it continues to follow a target that is still visible than when it has to reacquire a lost one. Severe occlusion, abrupt displacement, strong background interference, or accumulated drift can therefore cause the tracker to fail, and once that happens the original KCF framework offers almost no direct support for relocalization. This weakness remains a central issue

even in later improved variants^[5,19-20,22].

2.2 CSRT: Reliability-Aware Correlation Filtering

Lukežić et al.^[17] proposed CSRT to show that the basic correlation-filter framework can be made more reliable by modelling channel reliability and spatial reliability. Rather than assigning the same weight to every feature channel and image region, CSRT estimates which channels carry useful information and which spatial areas are trustworthy, and then uses these weights during tracking. The original paper is important because it demonstrates that better localisation quality can be obtained without abandoning the discriminative correlation-filter family altogether.

In practical terms, CSRT is often preferred when the tracking problem demands tighter localisation and greater stability than KCF can usually provide. This tends to matter most when the target changes appearance, when local occlusion interrupts the observation, or when the object boundary is not easy to represent cleanly. The price of that extra care is computational. Because CSRT relies on richer features and a more elaborate reliability mechanism, it runs more slowly than KCF. Its central trade-off is therefore straightforward: it gains robustness by giving up part of KCF’s speed advantage.

2.3 TLD: Long-Term Recovery Through Tracking, Learning, and Detection

The name TLD directly reflects its structure: tracking, learning, and detection are treated as three connected components rather than as one monolithic tracker. In the original formulation of Kalal et al.^[14], the tracking component is built on Median-Flow and uses optical-flow feature trajectories together with forward-backward error to indicate failure; that failure signal itself was formalised in an earlier study on forward-backward checking^[13]. Detection is handled through a sliding-window search supported by a classifier cascade, including variance filtering, random ferns, and nearest-neighbour verification. The learning component then updates the detector online through P-N learning with newly generated positive and negative samples.

What keeps TLD relevant in this comparison is its ability to search for the target again after tracking has already broken down. That matters in situations where occlusion lasts

for several frames, the object disappears temporarily, or the tracker has already drifted away from the correct region. In that sense, TLD addresses a part of the problem that short-term local trackers often leave exposed. The same design also creates the main practical difficulty. TLD is heavier than KCF, it runs more slowly, and it becomes much less comfortable in cluttered scenes where false detections or unstable recoveries are easier to trigger. This is why the method is more convincing as a recovery branch than as a continuously active high-speed tracker.

2.4 Why a Hybrid KCF–TLD Design Is Needed

The differences among KCF, CSRT, and TLD are real, but they do not remove the same practical bottleneck. In difficult UAV scenes, none of the three methods can comfortably hold speed, localisation quality, and post-failure recovery together for long. Occlusion, rapid motion, large scale variation, and heavy background interference can still destabilise all of them, even though the immediate cause of failure is not the same in each case. A second limitation is that their decisions remain dominated by short-term visual evidence, while longer-range motion regularity and scene-level context still play only a small role. That is the point at which a hybrid design becomes worth testing rather than merely describing.

The KCF–TLD proposal follows directly from that gap. KCF is fast enough to stay active frame by frame, but it gives little help once the target has genuinely been lost. TLD has the opposite attraction: it offers a chance to find the target again, yet it is too slow to dominate the whole sequence as the main tracker. The combination is therefore based on role assignment rather than constant coexistence. KCF keeps the ordinary tracking stage lightweight, while TLD is called only when KCF has become unstable or has already lost the target. Later UAV-oriented long-term tracking work moves in a comparable direction by keeping a fast correlation-filter core and supplementing it with stronger recovery cues^[9]. The hybrid is meant to solve a specific division of labour problem, not to merge two trackers for its own sake.

3 Evaluation

This project reached a clear intermediate result. It set up one framework in which KCF, CSRT, TLD, and the proposed KCF–TLD tracker could be compared on the same basis. The mixed concept is advanced from the description stage to the implementation stage and the benchmark data stage. That matters because the work shows the trade-off among speed, localisation quality, recovery ability, and engineering cost in one trial rather than as separate claims. The current KCF–TLD tracker is still weaker than CSRT in precision and stability, but the experiments are already enough to show which part of the design is promising and which part still causes the main loss.

The weak point is now fairly specific. The tracker still relies on local continuity checks and hand-set rules, so its decisions remain narrow when the scene becomes more complicated across time. Adjusting the present thresholds alone is unlikely to change that very much. A stronger next step would be to test better state estimation together with richer information sources, especially longer temporal context, stronger appearance modelling, scene structure, and semantic cues. If that additional information can be integrated well, the same KCF–TLD framework can be tested again as a more dependable tracker rather than only as a proof of concept.

4 Methodology

4.1 Overview of the Proposed Tracker

The proposed KCF–TLD tracker is organised around a practical division of labour rather than around constant parallel fusion. During the normal stage, the system keeps KCF in the leading role because the main engineering priority is still speed. TLD is held back most of the time, not because it is unimportant, but because allowing it to run continuously would erase the main benefit of using a correlation-filter tracker in the first place. Recovery is therefore treated as an exceptional state change instead of a permanent operating mode.

The algorithm proceeds from the first bound box of the first frame. We start at 1st frame. KCF is initialize first. The latest reliable target box is written as `last_bbox`. We return a failure counter, a pending recovery box, and a pending confirmation counter. At

most frame we run KCF and check whether the returned box is still reliable. If KCF is still consistent we accept the result and remove all recovery.

At the first frame, the initial bounding box is used to initialise KCF and to define the starting reliable state, namely the most recent reliable bounding box. From that point onward, the tracker also maintains a failure counter, a pending recovery box, and a confirmation counter for candidate recoveries. Most frames remain simple: KCF updates the target locally, the returned box is checked for continuity, and a stable result is accepted without further intervention. When that happens, the temporary recovery state is cleared immediately so that the system falls back into its cheapest operating pattern.

The control flow changes only when KCF starts to fail repeatedly or produces boxes that no longer look trustworthy. At that point, the algorithm enters recovery mode, reinitialises TLD from the last reliable target state, and queries it only on scheduled recovery frames. Any candidate produced by TLD is treated cautiously: it must survive overlap, plausibility, and multi-frame consistency checks before it is accepted as a genuine relocalization. Once that confirmation is achieved, KCF is reinitialised from the recovered box and the tracker returns to the fast path. The design goal is to leave the sequence in a lightweight KCF regime for as long as possible, then spend extra computation only when the probability of target loss has become too high.

Figure 1 gives a simplified system-level view before the detailed decision diagrams later in this chapter. Its purpose is only to show the main operating states and the direction of control transfer between them.

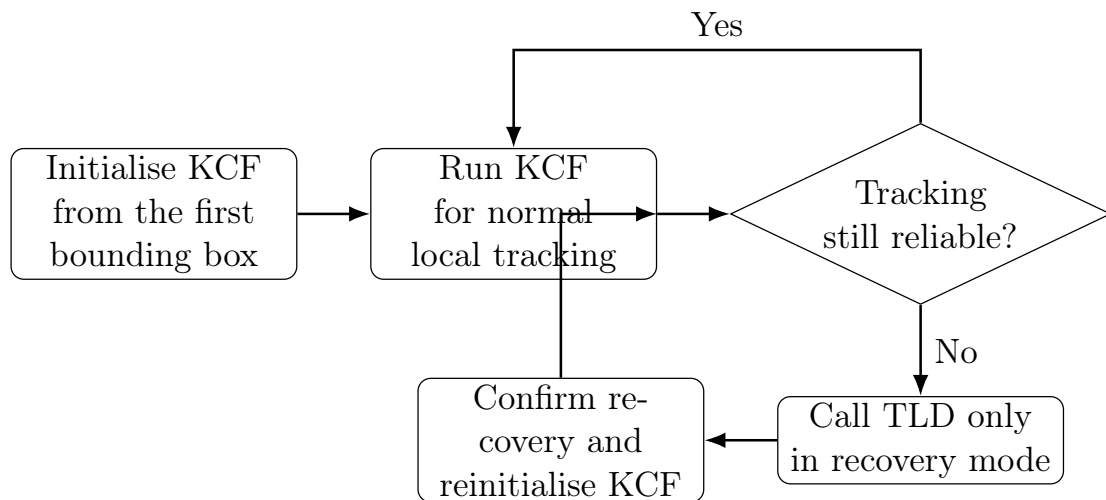


Figure 1: Simplified system-level flow of the proposed KCF-TLD tracker. The detailed threshold confirmation check is expanded later.

4.2 Tracking and Recovery Stages

4.2.1 Kernelized Correlation Filter stage

As the base tracker, KCF follows the standard frequency-domain regression formulation introduced by Henriques et al.^[12], which itself grows out of the earlier correlation-filter tradition represented by MOSSE^[4]. Given a sample patch x and a Gaussian-shaped response y , the correlation filter can be written as

$$\hat{w} = \frac{\hat{x}^* \odot \hat{y}}{\hat{x}^* \odot \hat{x} + \lambda},$$

where $\hat{x}$ and $\hat{y}$ are the Fourier transforms of the training sample and desired response, $\odot$ denotes element-wise multiplication, and λ is the regularisation term. In each frame, KCF evaluates the response map in a local region around the previous target state and selects the position with the highest response as the new prediction. This keeps the main tracker computationally light and highly responsive.

In the implemented system, KCF is not treated as reliable by default. Its output is checked against the most recent reliable bounding box by simple geometric continuity rules. In the current implementation, the KCF result is marked as unreliable when its IoU with that reliable box falls below 0.15, when the area ratio falls outside the range from 0.5 to 2.0, or when the box centre shifts too abruptly. The algorithm also waits for repeated failure instead of reacting to one temporary fluctuation. Recovery mode is triggered only when the failure streak reaches 2 frames. This keeps KCF responsible for fast local tracking while avoiding unnecessary recovery activation.

4.2.2 Tracking–Learning–Detection recovery stage

The TLD branch is introduced in order to raise the probability of finding the target again after KCF failure. In the hybrid design, TLD is not expected to carry long-term high-speed tracking. Its role is narrower: it acts as a relocalization mechanism once the KCF stage has become unstable. This design choice follows directly from the strengths and weaknesses identified in the literature review, and it is close in spirit to later UAV-specific efforts that keep a fast base tracker while supplementing it with stronger recovery cues^[9].

To prevent TLD from dominating the total runtime, recovery mode is deliberately restricted. TLD is initialized only when recovery begins, and it is not updated on every

frame thereafter. Instead, recovery is attempted only once every 3 frames in the current implementation. In addition, one recovery episode is limited to at most 120 frames. These restrictions are important because TLD is the main source of slowdown in difficult sequences. Without such limits, the hybrid would lose the speed advantage that motivates the use of KCF in the first place.

4.3 Thresholds, Decision Logic, and Detailed Flowcharts

The fusion part is not designed as a forced blending of two trackers. Instead, it separates the working regions in which each tracker is most useful. KCF handles the main tracking process, whereas TLD is responsible for recovery after loss. The key practical issue is therefore how to decide that KCF is no longer stable enough and how to verify that a TLD recovery box is credible.

In the current system, KCF is treated as unstable when the continuity between the current box and the previous valid box becomes too weak. The judgement is deliberately simple in practice. A box is rejected if its IoU with the previous valid box is below 0.15, and if the area ratio relative to the previous valid box moves outside 0.5 to 2.0. The values were chosen by small pilot tuning during implementation rather than a complete ablation study. A looser IoU floor delays recovery and gets more drift, and a much larger IoU floor makes it impossible to enter recovery. We see the same tradeoff between upper bounds that yield implausible scale jumps, and lower bounds that ignore legitimate target size fluctuations. The same is true if the failure streak is 2 frames, which prevents the system from responding too rapidly to one temporary fluctuation.

The next problem is how to judge whether the TLD recovery box is correct. What is the next problem to check if the TLD recovery box is valid? Again, it is better to use a cautious sequence of checks than a yes-or-no test. A candidate can only be accepted if it remains within 0.3 IoU of a previous one and is not immediately used. Instead, it must be confirmed over 2 valid recoveries and those two candidates must also coincide to at least 0.45 IoU. These values were chosen for the same practical reasons as instability limits: lower confirmation requires faster handover but more false recoveries, and stricter confirmation makes wrong acceptance less likely.

This multi-step confirmation is specially designed to avoid accepting an incorrect recovery box in situations where TLD generates a reasonable but unstable detection. The

threshold also reflects a strong awareness of runtime. Recovery mode is not allowed to call TLD continuously, and it cannot continue indefinitely. The recovery interval is 3 frames, and the maximum duration is 120 frames. The goal is to prevent the algorithm from staying stuck in a slow TLD-dominated state for too long.

4.3.1 Detailed main flowchart

Figure 2 summarises the overall logic of the proposed KCF–TLD tracker. A flowchart is more appropriate here than a long code listing, because the method is defined mainly by decision paths and state transitions rather than by implementation syntax.

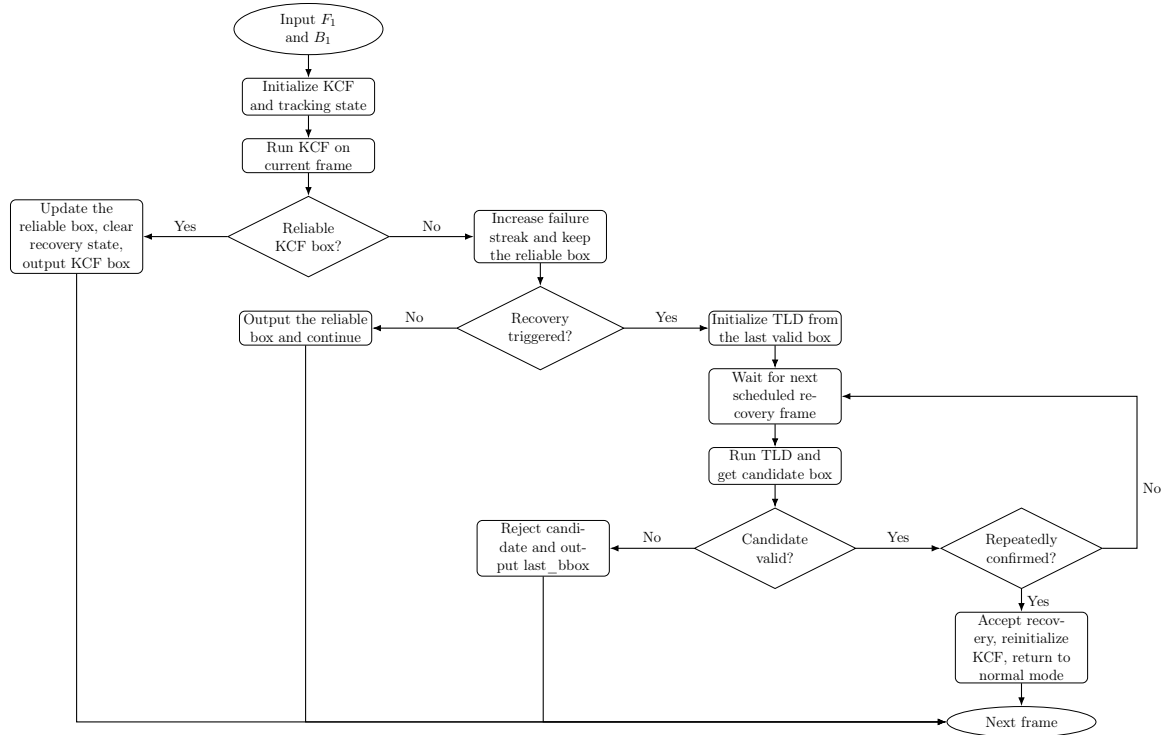


Figure 2: Overall flowchart of the proposed KCF–TLD relocation tracker.

4.3.2 Detailed recovery decision flow

Figure 3 isolates the recovery branch. This part is critical because the usefulness of the hybrid depends less on constant complexity than on whether recovery is triggered at sensible moments and whether doubtful TLD candidates are kept from being accepted too early.

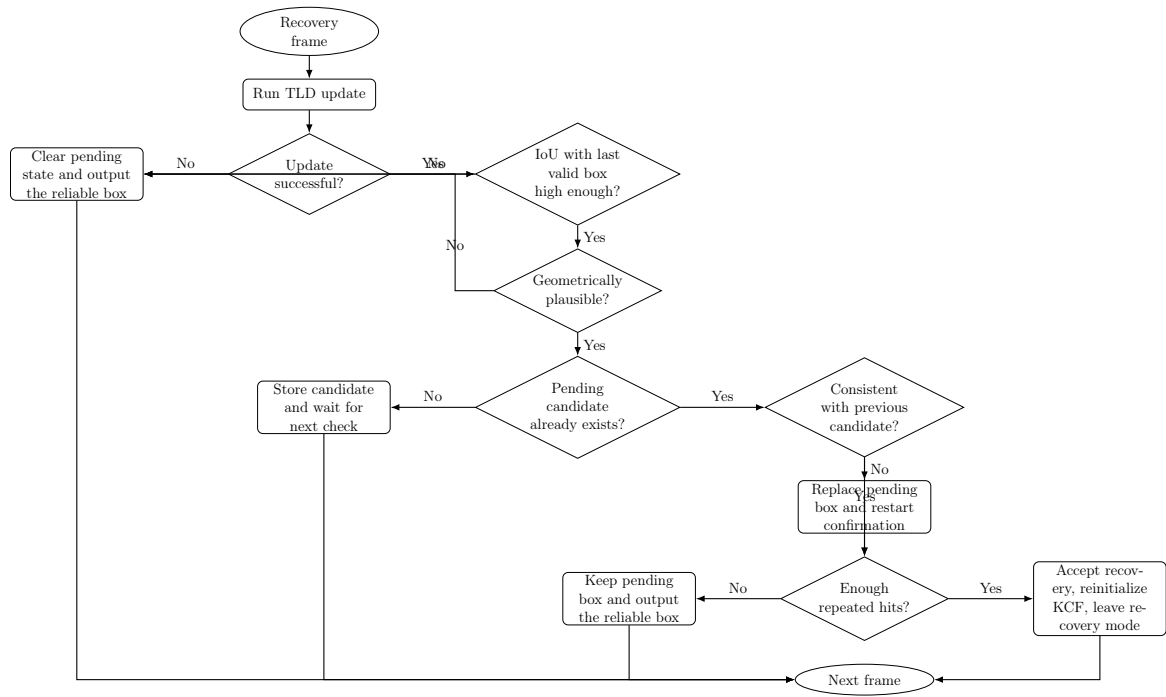


Figure 3: Detailed recovery decision flow for TLD candidate validation and KCF reinitialization.

5 Results and Analysis

5.1 Experimental Setup

The experiments were carried out on a directly comparable subset derived from UAV123. The tested sequences are *bike1* and *bike3*. To keep iterative debugging and validation manageable, the benchmark uses a frame step of 2, a maximum of 800 frames per sequence, and a resized evaluation resolution of 480×360 . The output includes IoU-related measures, localisation measures, stability indicators, efficiency indicators, diagnostic plots, and a leaderboard image.

To address repeatability more directly, each of the four algorithms was run 3 times under exactly the same benchmark settings. Table 1 reports the arithmetic mean of the aggregate metrics across those 3 repetitions. The benchmark plots shown later come from the third repeated run, because the overlap and localisation curves were visually the same across repetitions while the enlarged output remained easier to read than the earlier cropped figures.

5.2 Per-Sequence Interpretation

The two tested sequences reveal two different aspects of the present system. On *bike1*, the proposed KCF-TLD tracker does not improve over plain KCF under the current settings. In the representative third run, average IoU changes from 0.031499 for KCF to 0.030941 for KCF-TLD, while FPS drops from 313.91 to 155.28. By contrast, CSRT remains much stronger on the same sequence with an average IoU of 0.517839, and standalone TLD reaches 0.233849 while running more slowly than KCF.

On *bike3*, KCF-TLD matches KCF in average IoU at 0.067584, but FPS goes from 343.46 to 15.07 for the same run. This indicates that if the recovery branch is running in a difficult sequence, the runtime may become large without obtaining a localisation gain. CSRT still leads this sequence in localisation quality with average IoU 0.135511, while standalone TLD drops to 0.004608 and does not compete as a primary tracker here.

From the perspective of algorithm analysis, this contrast remains useful even though the hybrid does not win on the current subset. The main problem is not the existence of state switching itself, but the fact that the present recovery logic consumes substantial time while still failing to generate enough correct handovers. The current KCF-TLD

implementation should therefore be viewed as a prototype for recovery control rather than as a fully optimised final tracker.

5.3 Aggregate Comparison

Table 1 condenses the 3-run average into one view. CSRT is clearly the strongest tracker on this subset in terms of localisation quality, while KCF remains the fastest. Standalone TLD sits between them: it is far slower than KCF, but its average IoU and precision values remain above the current KCF and KCF–TLD implementations.

The more important point for the proposed method is that the present KCF–TLD configuration does not yet deliver a reliable net gain over plain KCF once the results are averaged across repeated runs. Average IoU drops slightly from 0.044169 to 0.043807, overlap AUC drops from 0.053110 to 0.052869, and centre precision remains unchanged at 0.111650. The only clear positive movement is tracking availability, which rises from 0.033981 to 0.050162. That increase indicates that the recovery branch can keep returning boxes more often, but the quality of those boxes is not yet strong enough to lift the main localisation metrics.

The cost is just as clear in the same table. Average FPS falls from 322.815 to 106.218, so the hybrid gives up much of the speed that makes plain KCF attractive in the first place. For UAV tracking that reduction matters directly, because runtime is part of whether the tracker remains usable at all. The current version therefore increases recovery activity, but it does not yet convert that extra computation into better overall localisation.

Table 1: Aggregate benchmark results on the current UAV123 subset, averaged across 3 repeated runs

Algorithm	Avg. IoU	AUC	Avg. CE	Prec.@20px	Norm. Prec.	Availability	Avg. FPS
KCF	0.044169	0.053110	83.531	0.111650	0.187702	0.033981	322.815
CSRT	0.383591	0.386427	15.408	0.826861	0.872168	1.000000	85.539
TLD	0.153355	0.159489	107.963	0.289644	0.422330	1.000000	55.805
KCF–TLD	0.043807	0.052869	98.853	0.111650	0.187702	0.050162	106.218

The repeated runs also show that the present benchmark is numerically stable on this small subset. Across the 3 repetitions, the aggregate quality metrics reported above remained unchanged to 6 decimal places for all four algorithms, whereas the visible variation appeared mainly in runtime. Table 2 therefore reports only the standard deviation of average IoU and average FPS, which is enough to indicate the current level of repeatability

without adding another large table.

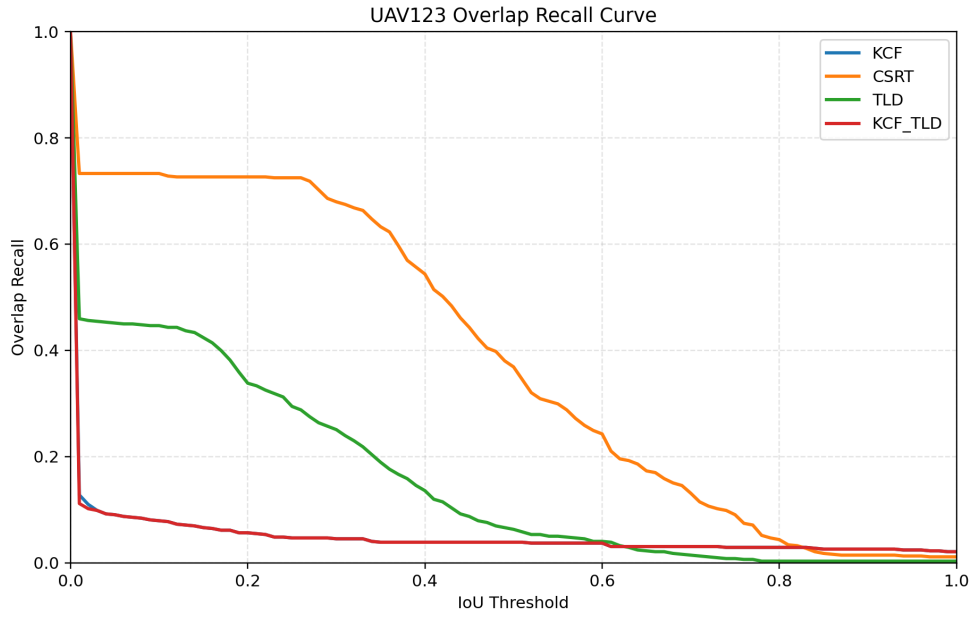
Table 2: Repeatability summary across the 3 repeated runs

Algorithm	SD of Avg. IoU	SD of Avg. FPS
KCF	0.000000	1.317
CSRT	0.000000	0.920
TLD	0.000000	0.691
KCF-TLD	0.000000	1.073

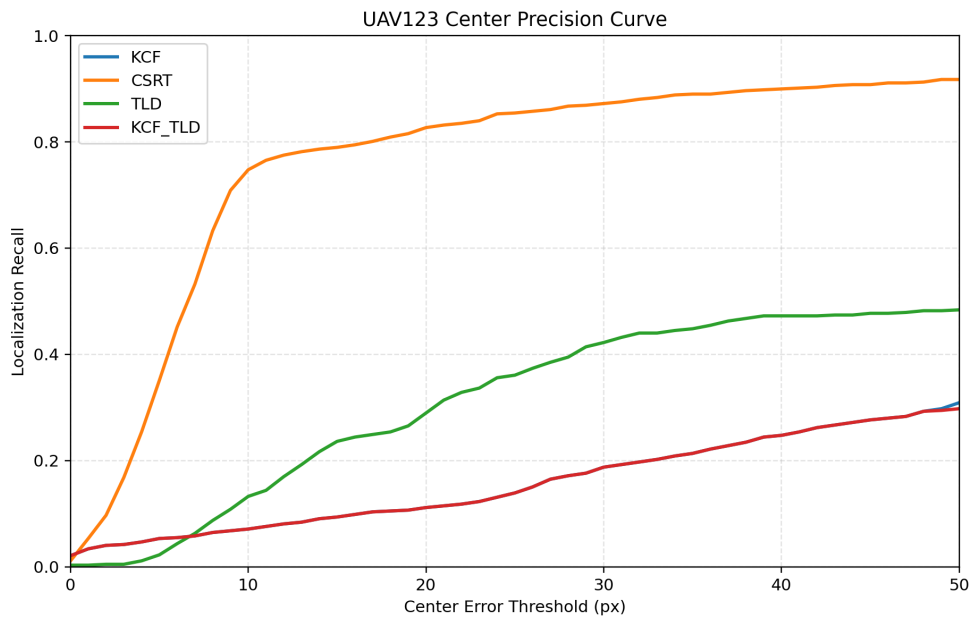
5.4 Benchmark Figures

Figures 4, 5, 6, and 7 are representative results of the third consecutive four-algorithm run. The result images were enlarged from the previous version so that the axis labels and legend text could be read.

Their function is analytical rather than decorative. The overlap and precision curves show that the four algorithms differ across a range of thresholds rather than at one convenient operating point. The grouped metric panels separate overlap quality, localisation accuracy, stability, and efficiency, making the trade-off easier to read than in the same large-scale table. The leaderboard image shows the same comparison in compressed format.

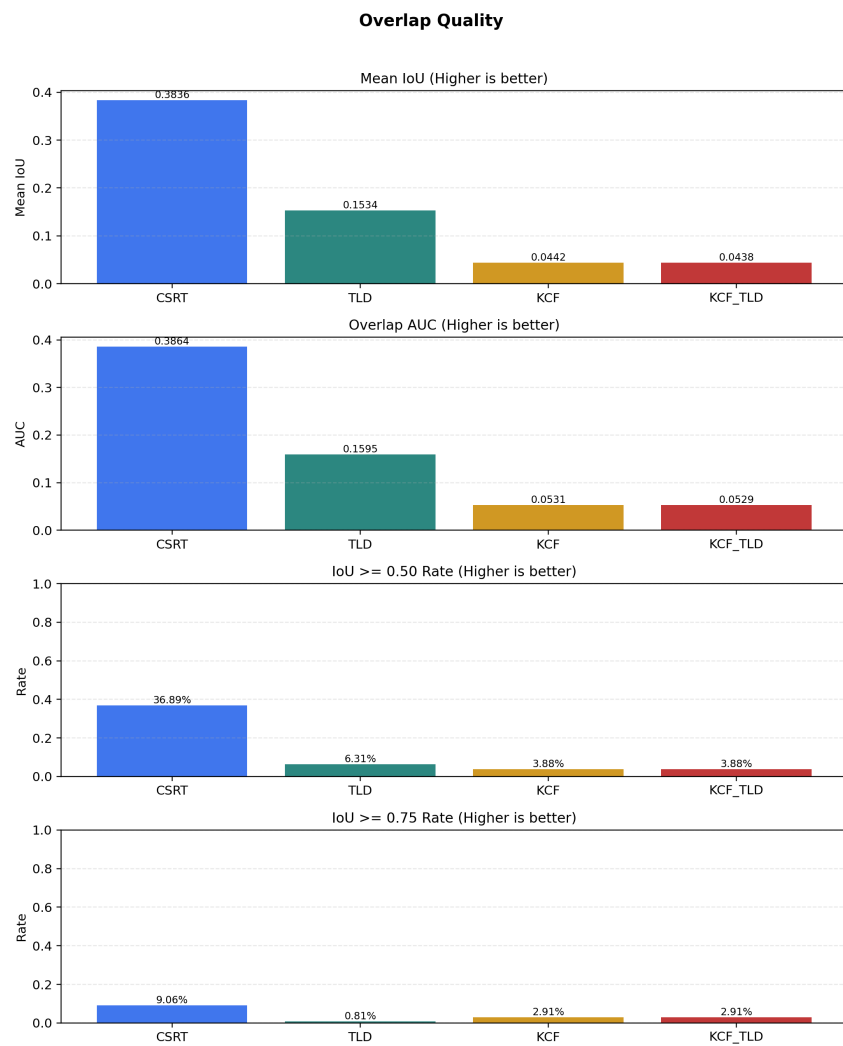


(a) Overlap recall curve

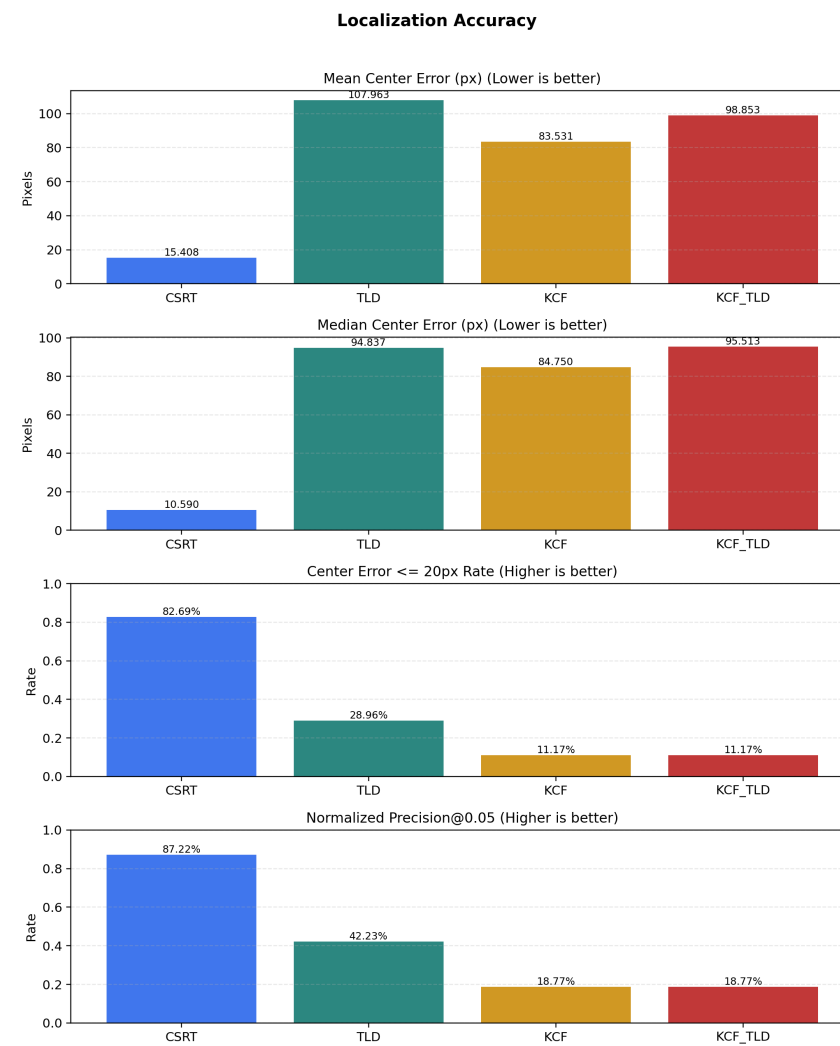


(b) Center precision curve

Figure 4: Representative overlap and center-precision curves from the third repeated four-algorithm benchmark run.

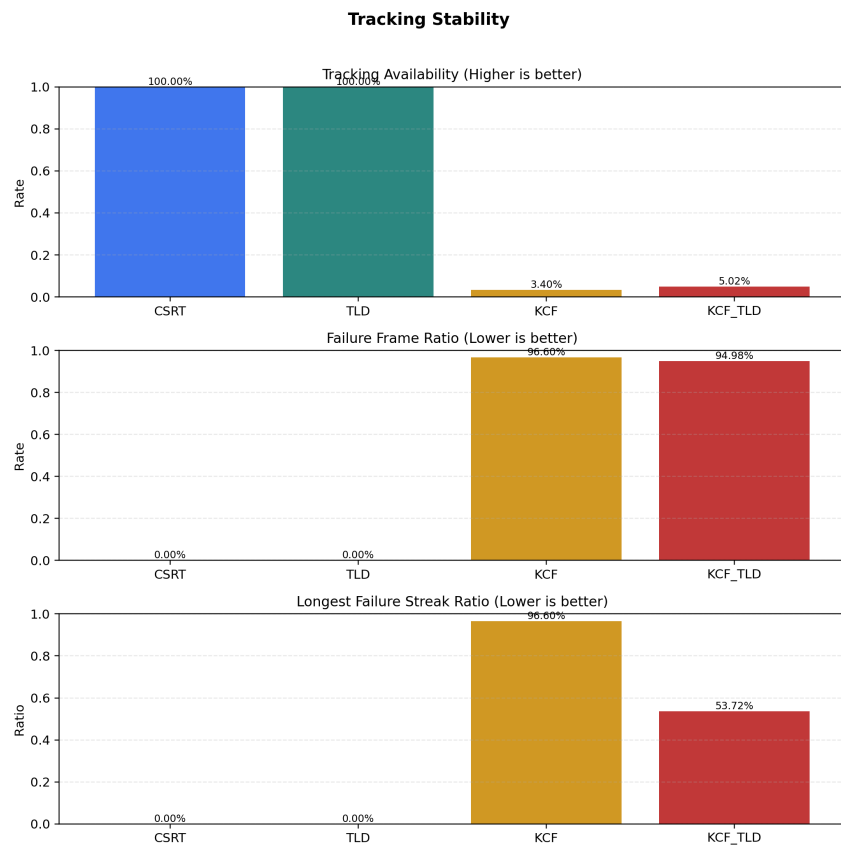


(a) Overlap quality metrics

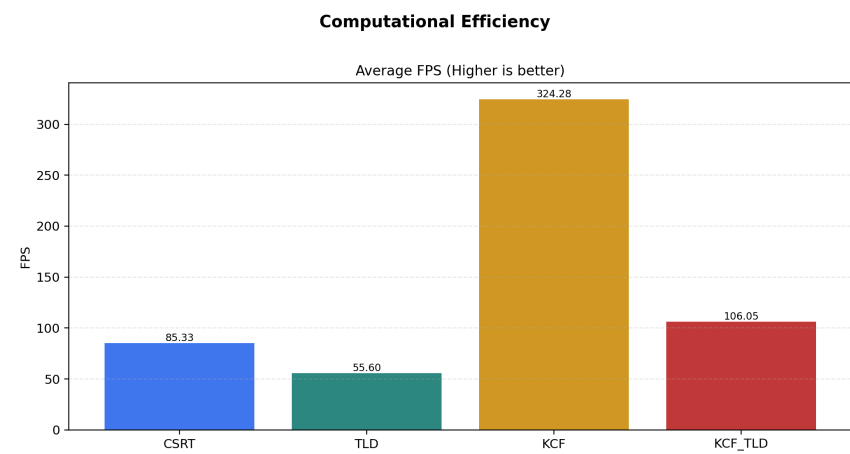


(b) Localization accuracy metrics

Figure 5: Enlarged overlap and localization panels from the third repeated run.



(a) Tracking stability metrics



(b) Computational efficiency metrics

Figure 6: Enlarged stability and efficiency panels from the third repeated run.

The figures point to the same pattern shown in the table. CSRT dominates the localisation-oriented panels, KCF dominates raw speed, and the present KCF-TLD tracker does not separate clearly from KCF on the quality curves even though it pays a visible efficiency penalty.

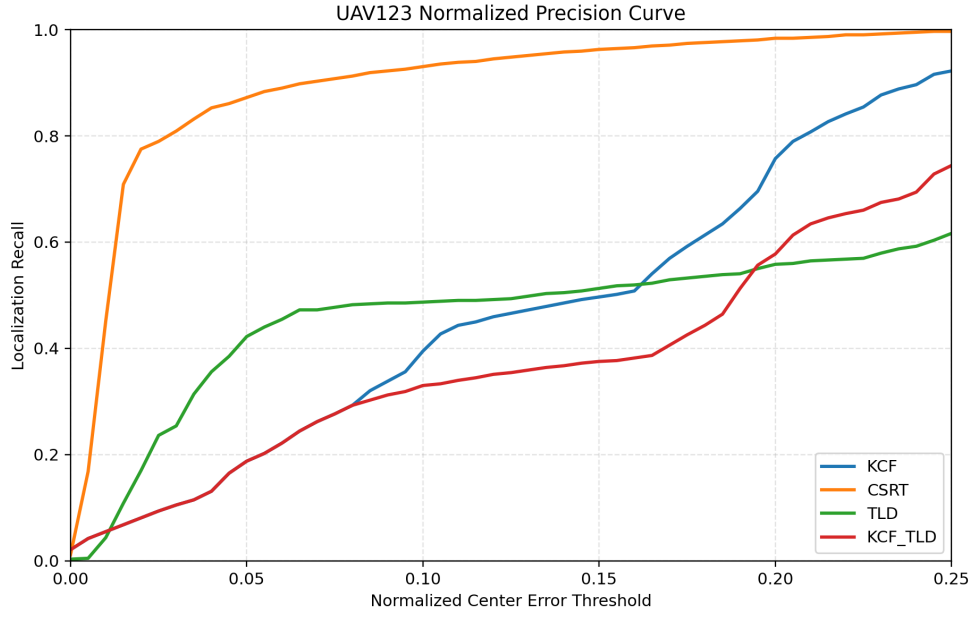
6 Discussion

The repeated experiments clarify the current position of KCF-TLD more precisely. At the present testing stage, KCF-TLD is weaker than CSRT in tracking accuracy, weaker than KCF in efficiency, and only slightly better than KCF in tracking availability. That means the current hybrid does not yet justify itself as a general-purpose improvement over the original tracker. The design idea is still meaningful, but the present recovery thresholds and confirmation logic are not sufficient to turn that idea into a competitive result.

The main advantage of the proposed method is that its design logic is easy to explain and closely tied to the strengths of the source algorithms. KCF is responsible for high-speed short-term tracking, while TLD is responsible for relocalization after failure. The two modules do not compete for the same task. Instead, they are assigned different functions. This still makes the hybrid easier to interpret and tune in engineering terms than a more opaque end-to-end design.

Relative to pure KCF, the hybrid also adds a more cautious protection mechanism. TLD is not triggered by one doubtful frame, and a recovery candidate is not accepted after one successful update. The recovery box must pass candidate filtering, consistency checking, and repeated confirmation before it is used to reinitialize KCF. These steps help reduce the chance of incorrect recovery in principle, but the present results suggest that the accepted recoveries are still too sparse or too weak to compensate for the extra runtime cost.

At the same time, the current method still has several clear limitations. First, the overall upper bound of the system is still strongly influenced by KCF, because ordinary tracking depends on KCF for most frames. Second, the failure judgment mechanism is based mainly on heuristic rules such as overlap, area change, and repeated failure. These rules are simple and interpretable, but their adaptability remains limited in scenes



(a) Normalized precision curve

Benchmark Leaderboard

Overlap Quality

Metric	Direction	CSRT	TLD	KCF	KCF_TLD
Mean IoU	Higher is better	0.3836	0.1534	0.0442	0.0438
Overlap AUC	Higher is better	0.3864	0.1595	0.0531	0.0529
IoU ≥ 0.50 Rate	Higher is better	36.89%	6.31%	3.88%	3.88%
IoU ≥ 0.75 Rate	Higher is better	9.06%	0.81%	2.91%	2.91%

Localization Accuracy

Metric	Direction	CSRT	TLD	KCF	KCF_TLD
Mean Center Error (px)	Lower is better	15.408	107.963	83.531	98.853
Median Center Error (px)	Lower is better	10.590	94.837	84.750	95.513
Center Error ≤ 20 px Rate	Higher is better	82.69%	28.96%	11.17%	11.17%
Normalized Precision@0.05	Higher is better	87.22%	42.23%	18.77%	18.77%

Tracking Stability

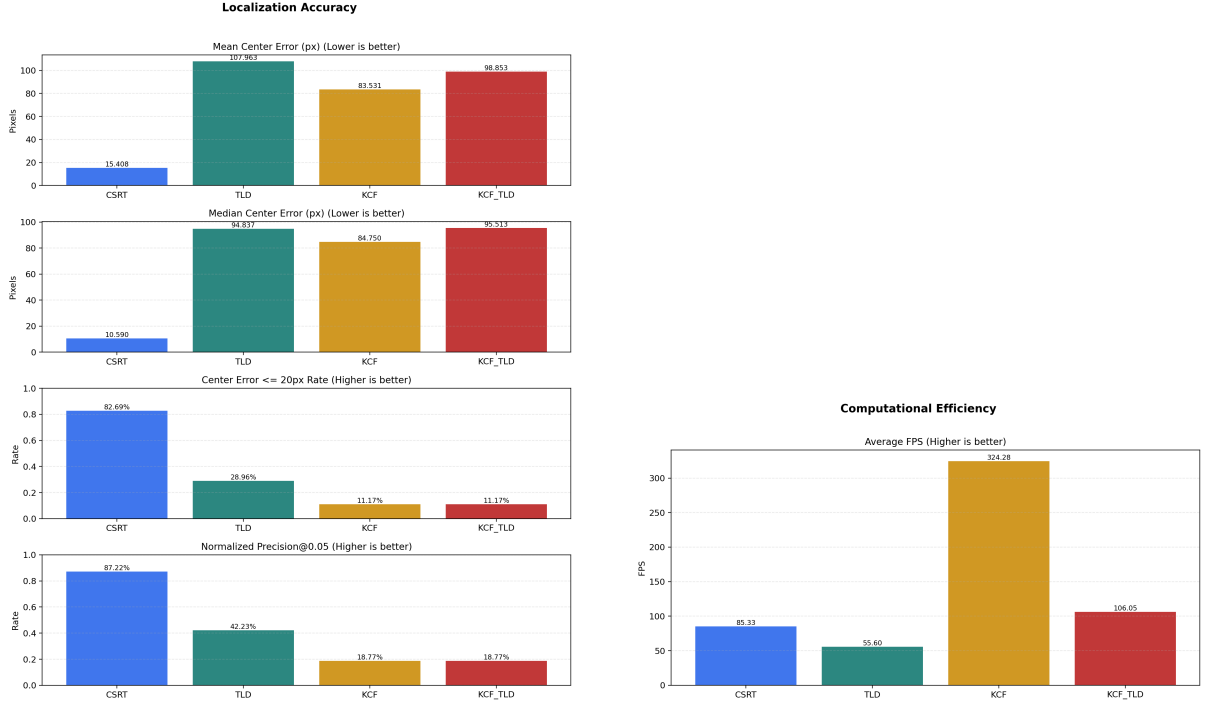
Metric	Direction	CSRT	TLD	KCF	KCF_TLD
Tracking Availability	Higher is better	100.00%	100.00%	3.40%	5.02%
Failure Frame Ratio	Lower is better	0.00%	0.00%	96.60%	94.98%
Longest Failure Streak Ratio	Lower is better	0.00%	0.00%	96.60%	53.72%

Computational Efficiency

Metric	Direction	CSRT	TLD	KCF	KCF_TLD
Average FPS	Higher is better	85.33	55.60	324.28	106.05

(b) Benchmark leaderboard

Figure 7: Additional representative outputs showing normalized precision and the aggregate leaderboard for the third repeated run.



(a) Localization comparison in the third repeated four-algorithm run

(b) Efficiency comparison in the third repeated four-algorithm run

Figure 8: Representative localisation and efficiency panels from the third repeated run. CSRT remains strongest in localisation quality, whereas KCF remains the fastest method.

involving long occlusion, strong clutter, rapid motion, or strong non-linear change. Third, the recovery ability of TLD is not always stable, and the geometric constraints used here may also reduce the chance of successful recovery in large-range search situations. A further caution is that some stability-style indicators, especially tracking availability for standalone TLD, can look better than the actual localisation quality because the detector may keep outputting boxes even when those boxes are not accurate.

7 Conclusion

This paper examined KCF, CSRT, and TLD as three representative discriminative tracking algorithms for UAV-style visual tracking tasks, and compared them from the perspective of principle, behaviour, and practical use. The comparison shows that KCF is attractive because of its very high speed, CSRT is stronger in localisation reliability and robustness, and TLD is more valuable in the stage after failure because of its explicit redetection ability. These differences explain why no single one of the three methods can easily satisfy speed, accuracy, and recovery at the same time.

On that basis, the paper proposed a KCF–TLD hybrid tracker. The method uses KCF to preserve high-speed tracking during the normal stage, and activates TLD only when KCF becomes unstable or loses the target. A multi-frame confirmation mechanism is then used to make the accepted recovery box more reliable before KCF is reinitialized. The repeated experiments show that this “high-speed main tracker plus auxiliary recovery module” design is implementable, but the present configuration does not yet improve the main localisation metrics relative to pure KCF once the results are averaged across 3 runs. Its current effect is closer to a limited increase in recovery activity than to a finished accuracy improvement.

That result should be read as a prototype outcome, not as a finished tracker. The current system still leans on heuristic thresholds and hand-crafted recovery decisions, and those choices limit its behaviour in harder UAV sequences. The next improvement is not to add complexity everywhere, but to make the recovery decision itself better informed. Better state assessment, stronger modelling of motion continuity and appearance consistency, and wider scene context are the most direct places to test. If those parts improve, the tracker should be judged again on whether it can recover more reliably without losing too much of the speed that motivated the hybrid in the first place.

References

- [1] BERTINETTO L, VALMADRE J, GOLODETZ S, MIKSIK O, TORR P H S. Staple: Complementary Learners for Real-Time Tracking[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 1401-1409. DOI: [10.1109/CVPR.2016.156](https://doi.org/10.1109/CVPR.2016.156).
- [2] BERTINETTO L, VALMADRE J, HENRIQUES J F, VEDALDI A, TORR P H S. Fully-Convolutional Siamese Networks for Object Tracking[C/OL]//Proceedings of the European Conference on Computer Vision (ECCV) Workshops. 2016: 850-865. DOI: [10.1007/978-3-319-48881-3_56](https://doi.org/10.1007/978-3-319-48881-3_56).
- [3] BIBI A, GHANEM B. Multi-Template Scale-Adaptive Kernelized Correlation Filters[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops. 2015: 613-620. DOI: [10.1109/ICCVW.2015.85](https://doi.org/10.1109/ICCVW.2015.85).
- [4] BOLME D S, BEVERIDGE J R, DRAPER B A, LUI Y M. Visual Object Tracking

- Using Adaptive Correlation Filters[C/OL]//2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. 2010: 2544-2550. DOI: [10.1109/CVPR.2010.5539960](#).
- [5] CHEN X, YANG Z, YU W. An Improved KCF Mixed with Deep Feature for Pedestrian Tracking[J/OL]. Journal of Physics: Conference Series, 2023, 2465(1): 012015. DOI: [10.1088/1742-6596/2465/1/012015](#).
- [6] DANELLJAN M, BHAT G, KHAN F S, FELSBURG M. ECO: Efficient Convolution Operators for Tracking[C/OL]//2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017: 6931-6939. DOI: [10.1109/CVPR.2017.733](#).
- [7] DANELLJAN M, BHAT G, KHAN F S, FELSBURG M. ATOM: Accurate Tracking by Overlap Maximization[C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 4660-4669. DOI: [10.1109/CVPR.2019.00479](#).
- [8] DANELLJAN M, HAGER G, KHAN F S, FELSBURG M. Learning Spatially Regularized Correlation Filters for Visual Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2015: 4310-4318. DOI: [10.1109/ICCV.2015.490](#).
- [9] FAN J, SHEN X, WU Y. Long-Term Visual Tracking Algorithm for UAVs Based on Kernel Correlation Filtering and SURF Features[J/OL]. The Visual Computer, 2022, 38(3): 1043-1057. DOI: [10.1007/s00371-021-02103-4](#).
- [10] GAO J, ZHANG T, YANG X, XU C. Deep Relative Tracking[J/OL]. IEEE Transactions on Image Processing, 2019, 28(12): 5987-6002. DOI: [10.1109/TIP.2019.2924568](#).
- [11] GUO Q, FENG W, ZHOU C, HUANG R, WAN L, WANG S. Learning Dynamic Siamese Network for Visual Object Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 1781-1789. DOI: [10.1109/ICCV.2017.196](#).
- [12] HENRIQUES J F, CASEIRO R, MARTINS P, BATISTA J. High-Speed Tracking with Kernelized Correlation Filters[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2015, 37(3): 583-596. DOI: [10.1109/TPAMI.2014.2345390](#).

- [13] KALAL Z, MIKOLAJCZYK K, MATAS J. Forward-Backward Error: Automatic Detection of Tracking Failures[C/OL]//2010 20th International Conference on Pattern Recognition. 2010: 2756-2759. DOI: [10.1109/ICPR.2010.675](https://doi.org/10.1109/ICPR.2010.675).
- [14] KALAL Z, MIKOLAJCZYK K, MATAS J. Tracking-Learning-Detection[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2012, 34(7): 1409-1422. DOI: [10.1109/TPAMI.2011.239](https://doi.org/10.1109/TPAMI.2011.239).
- [15] KRISTAN M, MATAS J, LEONARDIS A, et al. The Seventh Visual Object Tracking VOT2019 Challenge Results[C/OL]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops. 2019: 2206-2241. DOI: [10.1109/ICCVW.2019.00278](https://doi.org/10.1109/ICCVW.2019.00278).
- [16] LI Y, ZHU J, HOI S C H. Reliable Patch Trackers: Robust Visual Tracking by Exploiting Reliable Patches[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015: 353-361. DOI: [10.1109/CVPR.2015.7298632](https://doi.org/10.1109/CVPR.2015.7298632).
- [17] LUKEŽIĆ A, VOJÍŘ T, ZAJC L Č, MATAS J, KRISTAN M. Discriminative Correlation Filter with Channel and Spatial Reliability[C/OL]//2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017: 4847-4856. DOI: [10.1109/CVPR.2017.515](https://doi.org/10.1109/CVPR.2017.515).
- [18] SMEULDERS A W M, CHU D M, CUCCHIARA R, CALDERARA S, DEGHAN A, SHAH M. Visual Tracking: An Experimental Survey[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2014, 36(7): 1442-1468. DOI: [10.1109/TPAMI.2013.230](https://doi.org/10.1109/TPAMI.2013.230).
- [19] TANG D, JIN W, LIU D, CHE J, YANG Y. Siam Deep Feature KCF Method and Experimental Study for Pedestrian Tracking[J/OL]. Electronics, 2023, 12(3): 669. DOI: [10.3390/electronics12030669](https://doi.org/10.3390/electronics12030669).
- [20] WANG C, ZHANG L, XIE L, YUAN J. Kernelized Correlation Filter Tracking with Scale Adaptation and Spatial Reliability[J/OL]. IEEE Access, 2018, 6: 50748-50758. DOI: [10.1109/ACCESS.2018.2869655](https://doi.org/10.1109/ACCESS.2018.2869655).
- [21] ZHANG K, ZHANG L, LIU Q, ZHANG D, YANG M H. Fast Visual Tracking via Dense Spatio-Temporal Context Learning[C/OL]//Proceedings of the European

- Conference on Computer Vision (ECCV). 2014: 127-141. DOI: [10.1007/978-3-319-10602-1_9](https://doi.org/10.1007/978-3-319-10602-1_9).
- [22] ZHANG L, SUGANTHAN P N. Visual Tracking with Convolutional Neural Network and Kernelized Correlation Filters[J/OL]. Journal of Visual Communication and Image Representation, 2017, 45: 174-183. DOI: [10.1016/j.jvcir.2017.03.003](https://doi.org/10.1016/j.jvcir.2017.03.003).